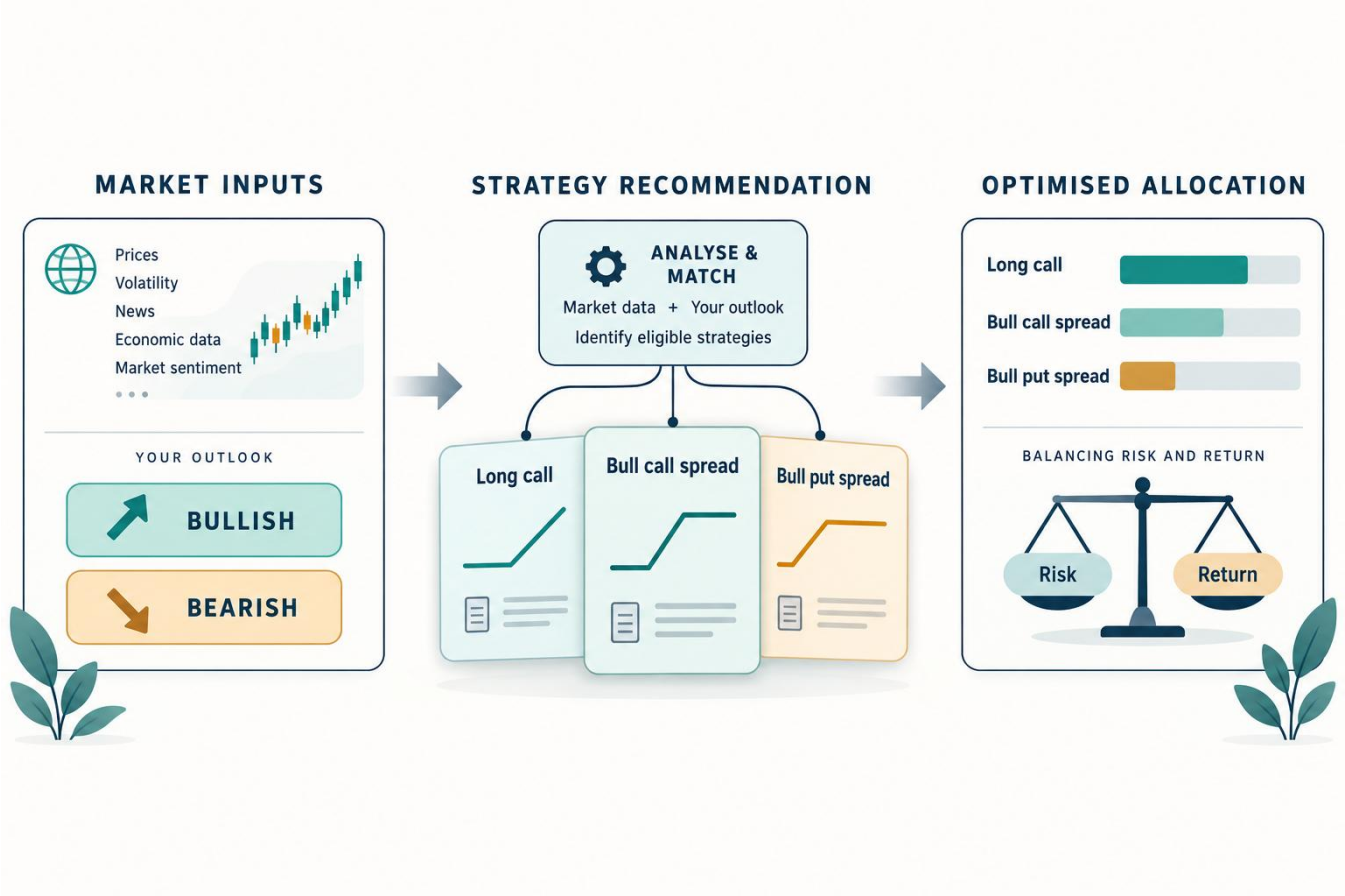
NUS-ISS Master of Technology in Artificial Intelligence Systems



Options Recommendation and Optimization System

Full Name	Group ID	Student ID	Email
Tey Hao Teck	3	A0362588M	teyhaoteck@u.nus.edu

Table of Contents

1. Executive Summary 4

2. Business Problem Background and Objectives 5

2.1 Business Background 5
2.2 Problem Statement 5
2.2.1 Complexity of Options Strategy Selection 5
2.2.2 Competing Return and Risk Objectives 6
2.3 Project Objectives 6
2.4 Scope of the Project 6
2.5 Expected Contributions 7

3. Dataset 8

3.1 Data Sources 8
3.1.1 Yahoo Finance Market Data 8
3.1.2 Historical SPY Options Data 9
3.2 Dataset Limitations 9

4. Data Preparation and Feature Engineering 10

4.1 Data Validation and Standardization 10
4.2 Options Data Filtering 10
4.3 Underlying Trend and Momentum 11
4.4 Volatility and Expected Price Movement 12
4.5 Liquidity and Options Features 13
4.6 Quality Checks and Prepared Outputs 14

5. System Architecture and Model Design 15

5.1 System Architecture 15
5.2 Supported Options Strategy 16
5.2.1 Long Call 16
5.2.2 Bull Call Spread 16
5.2.3 Bull Put Spread 17
5.2.4 Long Put 17
5.2.5 Bear Put Spread 17
5.2.6 Bear Call Spread 18
5.2.7 Overview of each strategy 18
5.3 Knowledge Graph and Rule-Based Strategy Reasoning 19
5.3.1 Knowledge Graph Representation 19
5.3.2 Direction and Volatility Rules 20

5.3.3 Market Condition Adjustments 20
5.3.4 Strategy Ranking and Explanation 21

6. Strategy Optimization and Evaluation 21

6.1 Strategy Payoff Evaluation 22
6.2 Monte Carlo Outcome Estimation 22
6.3 Transaction Cost Adjustment 24
6.4 Multi-Objective Fitness Function 24
6.5 Genetic Algorithm Optimization 25
6.6 Evaluation 26

7. End-to-End Recommendation Application 28

7.1 User Input and Pipeline Execution 28
7.2 Recommendation Assembly and Validation 29
7.3 Recommendation and Market Context 29
7.4 Payoff Visualisation and Alternatives 30
7.5 System Evidence and Limitation 31

8. System Evaluation through Historical Backtesting 32

8.1 Backtesting Methodology 32
8.2 Chronological Trading Policy 32
8.2.1 Trend Alignment 33
8.2.2 Previously Settled Strategy Evidence 33
8.3 Risk Management and Trade Simulation 33
8.4 Overall Backtesting Results 34

9. Future Improvements 36

9.1 Expand Market Data and Strategy Coverage 36
9.2 Strengthen the Recommendation Rules 37
9.3 Improve Outcome Estimates and Uncertainty Reporting 37

10. Conclusion 38

11. References 38

1. Executive Summary

Selecting an appropriate options strategy requires investors to consider their market outlook alongside volatility, liquidity, contract expiry, strike prices and potential losses. Multiple strategies can express the same bullish or bearish view yet produce substantially different outcomes. Evaluating these alternatives manually can be time-consuming and inconsistent, particularly when comparing individual contracts and balancing expected returns against risk.

This project develops an Options Strategy Recommendation and Optimisation System that combines market data analysis, knowledge representation, rule-based reasoning and genetic optimisation. Users provide a stock ticker and a bullish or bearish outlook. The system then retrieves historical underlying prices and current option chains from Yahoo Finance, prepares the data and calculates features describing market trend, momentum, volatility, expected price movement and contract liquidity.

A knowledge graph represents relationships between the user's outlook, observed market conditions and supported strategy families. Explicit rules rank six directional strategies: Bull Call Spread, Bull Put Spread, Long Call, Bear Put Spread, Bear Call Spread and Long Put. The reasoning process produces explanations for the rankings, allowing users to understand how volatility, trend alignment and liquidity influence the recommendation.

The payoff engine constructs valid combinations of option contracts and calculates their expiration payoffs, maximum losses and breakeven prices. Monte Carlo simulation estimates expected profit and probability of profit. A genetic algorithm searches the resulting candidate universe using a weighted fitness function that balances cost-adjusted expectancy, probability of profit, downside protection, capital efficiency, liquidity and rule alignment. The selected recommendation specifies the option legs, expiry and strikes, together with its estimated risk, costs and supporting explanations.

System evaluation includes controlled rule scenarios, comparisons with baseline selection methods, repeated genetic algorithm runs, and sensitivity analysis of transaction costs and fitness weights. Historical underlying returns also provide stress scenarios for current recommendations. A separate historical backtesting notebook evaluates simulated SPY options trades using quotes from 2020 to 2022. This evaluation adds a chronological selection policy based on trend alignment, previously settled strategy outcomes and risk limits. Its results are retrospective, and the default expiration price uses the dataset's underlying snapshot as a proxy rather than an independently verified official closing price.

A Streamlit application integrates the live recommendation pipeline into an interactive interface. It presents the selected strategy, exact contracts, payoff profile, alternative candidates and evaluation evidence. The project demonstrates how intelligent reasoning and optimisation can support a structured, explainable options selection process. It provides decision support while recognising that model assumptions, changing market conditions and execution uncertainty limit the reliability of estimated outcomes.

2. Business Problem Background and Objectives

2.1 Business Background

Options provide investors with several ways to express a directional market view. An investor expecting a stock price to rise may consider a Long Call, Bull Call Spread or Bull Put Spread. Similarly, a bearish view may be expressed through a Long Put, Bear Put Spread or Bear Call Spread. Although these strategies support the same broad direction, their premiums, payoff structures and risk characteristics differ.

Choosing a strategy therefore requires more than identifying whether the market outlook is bullish or bearish. Investors must also consider implied volatility, historical price behaviour, contract liquidity, time to expiry and the relationship between strike prices and the current underlying price. After selecting a strategy family, they must identify the specific contracts that form the position.

These decisions involve competing objectives. A candidate with a higher estimated probability of profit may offer a smaller potential reward, while another with greater expected profit may require more capital or expose the investor to a larger potential loss. A systematic decision-support process can help users compare these trade-offs consistently and understand the reasoning behind a recommendation.

2.2 Problem Statement

2.2.1 Complexity of Options Strategy Selection

Several options strategies can support the same directional outlook. However, each strategy responds differently to market movements and volatility conditions. Users must assess these differences before determining which approach is suitable for their expectations.

Without a structured reasoning framework, strategy selection may depend heavily on personal familiarity or subjective judgement. This can lead to inconsistent decisions and make it difficult to explain why one strategy was preferred over another.

2.2.2 Competing Return and Risk Objectives

No single performance measure captures every aspect of a strategy's suitability. Maximizing expected profit alone may favor strategies with greater potential losses, while prioritizing probability of profit may overlook the magnitude of those losses. Transaction costs and liquidity also influence the overall assessment.

The system must therefore evaluate strategies using several objectives. These objectives should be combined transparently so that users can understand how estimated return, downside protection, capital efficiency and liquidity contribute to the recommendation.

2.3 Project Objectives

The aim of this project is to develop an explainable options strategy recommendation and optimisation system for a user-specified stock ticker and directional outlook. The specific objectives are to:

1. Automate market-data preparation. Retrieve and validate historical underlying prices and current options data for analysis.
2. Derive interpretable market features. Calculate indicators describing trend, momentum, volatility, expected price movement and liquidity.
3. Represent options-strategy knowledge. Use a knowledge graph to organise relationships between user outlook, market conditions and supported strategy families.
4. Apply explainable reasoning rules. Rank strategies according to directional suitability, volatility conditions, trend alignment and liquidity, with supporting explanations.
5. Evaluate strategy outcomes. Calculate payoff characteristics and estimate expected profit, probability of profit and downside exposure.
6. Optimise strategy recommendations. Apply a genetic algorithm using a weighted fitness function that balances cost-adjusted expectancy, probability of profit, downside protection, capital efficiency, liquidity and rule alignment.
7. Evaluate and demonstrate the system. Compare selection methods, examine sensitivity to assumptions, conduct historical evaluation and present recommendations through an interactive application.

2.4 Scope of the Project

The live application analyses one stock ticker and one bullish or bearish outlook per request. The outlook is supplied by the user, while the system evaluates observed market conditions to support its strategy recommendation.

The project covers six directional options strategies:

Bullish	Bearish
Bull Call Spread	Bear Put Spread
Bull Put Spread	Bear Call Spread
Long Call	Long Put

The recommendation pipeline includes data preparation, feature engineering, knowledge representation, rule-based reasoning, payoff evaluation and genetic optimisation. The Streamlit application presents the recommended strategy, supporting explanations, payoff profile, alternatives and evaluation evidence.

The live evaluation uses current options data, simulated terminal prices and historical-underlying stress scenarios. A separate historical evaluation uses SPY options data from 2020 to 2022. This evaluation applies additional trend, settled evidence and risk filters that are separate from the live recommendation process.

Neutral strategies, automated trading, early exercise and assignment modelling, and complete portfolio management are outside the implemented scope. The system provides decision support and does not guarantee future investment performance.

2.5 Expected Contributions

The project integrates market-data analysis, knowledge representation, rule-based reasoning and genetic optimisation into a structured options strategy recommendation process. It provides a repeatable method for assessing strategies against a user's outlook and observed market conditions.

The reasoning component explains why particular strategy families are suitable, while the optimisation component balances multiple evaluation objectives. This combination helps users understand both the basis of the recommendation and the trade-offs involved.

The project also establishes an evaluation framework covering rule behaviour, optimisation consistency, sensitivity to assumptions and retrospective trading outcomes. The interactive application makes these results accessible and supports transparent comparison of the recommended strategy with available alternatives.

3. Dataset

3.1 Data Sources

The project uses two main data sources: Yahoo Finance for the live recommendation pipeline and a historical SPY options dataset for retrospective system evaluation. These sources serve different purposes. The live data supports recommendations under current market conditions, while the historical dataset allows the system to be evaluated using observations from earlier periods.

3.1.1 Yahoo Finance Market Data

The data-collection notebook retrieves market information from Yahoo Finance through the yfinance Python library.[1] By default, it downloads two years of daily underlying-price history for the selected ticker, together with the available option chains for the chosen expiry range.

The main data categories are summarized below.

Data Category	Main Fields	Purpose
Historical Underlying Prices	Date, Open, High, Low, Close and Volume	Calculate trend, momentum and historical volatility
Current Options Data	Option type, Expiry date, Strikes, Bis, Ask, Last Price and Implied Volatility	Support strategy payoff modeling and volatility analysis
Options Activity	Trading Volume and Open Interest	Access Liquidity

Both call and put data are retained so that the same prepared dataset can support bullish and bearish strategy analysis. The user's directional outlook is introduced at the reasoning stage rather than during data collection.

The notebook supports either a single ticker or batch collection for the current S&P 500 constituents. In batch mode, the constituent list is retrieved from Wikipedia, and the ticker symbols are standardized for Yahoo Finance. The Streamlit application uses the single-ticker mode and retrieves fresh data for each recommendation request.

Each successful collection produces historical-price features, raw options data, a prepared options dataset, a market snapshot and a data-quality report. These outputs provide a consistent input for subsequent feature engineering and reasoning.

3.1.2 Historical SPY Options Data

The historical evaluation uses the SPY Daily EOD Options Quotes 2020–2022 dataset from Kaggle [2], stored locally as “spy_2020_2022.csv”. SPY is the underlying instrument used throughout this backtesting stage.

The dataset contains dated call and put observations, including quote timestamps, underlying prices, expiry dates, days to expiry, strikes, bid and ask prices, displayed quote sizes, volume, implied volatility and option sensitivities.

The backtesting notebook organizes these records into daily snapshots. It uses earlier underlying-price observations to calculate market features and generates recommendations at weekly decision points after an initial warm-up period. Subsequent-session quotes are used to simulate entry, while the underlying price recorded on the expiry date provides the default proxy for calculating expiration payoff.

3.2 Dataset Limitations

The dataset has several limitations that should be considered when interpreting the results:

1. The historical options data is limited to SPY from 2020–2022 and may not represent every type of market condition or economic cycle.
2. Options data may contain missing values, low trading activity, zero bids or wide bid–ask spreads, which can affect strategy evaluation.
3. Results obtained using SPY may not generalize effectively to individual stocks or instruments operating in other markets.
4. Current information retrieved from Yahoo Finance depends on data availability, quote freshness and retrieval reliability.
5. Open interest is unavailable in the historical dataset, requiring liquidity assessment to rely on trading volume and bid–ask spread quality.
6. Historical underlying-price snapshots are used as proxies for expiration prices and may differ from official closing prices.
7. Recorded bid and ask prices may not reflect achievable execution prices, causing simulated outcomes to differ from actual trading results.

4. Data Preparation and Feature Engineering

The raw market data is prepared and transformed into interpretable features before entering the reasoning and optimization stages. These procedures are implemented in the "01_Data_Collection_and_Preparation.ipynb" and "02_Market_Feature_Engineering.ipynb" notebooks. The preparation process validates historical prices and options data, filters unsuitable observations and calculates indicators describing market conditions. The resulting datasets provide a consistent foundation for options strategy recommendation.

4.1 Data Validation and Standardization

The system standardizes ticker symbols and converts numerical fields into suitable data types. Historical price observations are arranged chronologically, while option expiry dates are converted into a consistent format.

The validation process checks that historical prices are available, the latest underlying price is positive and future option expiries exist. It also verifies that the prepared options dataset contains both calls and puts and that option identifiers are unique.

For valid bid and ask quotes, the midpoint is calculated as

$$Mid\ Price = \frac{Bid\ Price + Ask\ Price}{2}$$

```
result["mid_price"] = np.where(
    valid_quote,
    (result["bid"] + result["ask"]) / 2,
    result["lastPrice"],
)
```

A valid quote requires a non-negative bid, a positive ask and an ask price greater than or equal to the bid. When these conditions are not satisfied, the last traded price is used as a fallback midpoint estimate. These observations remain subject to subsequent filtering and quality checks.

4.2 Options Data Filtering

The data-collection stage primarily retrieves options with 21 to 60 calendar days remaining until expiry. If no expiry falls within this range, the nearest available future expiry to the range's midpoint is used.

The initial filtering process retains observations with positive midpoint prices, implied volatility between 0.01 and 5.00, and strike prices between 50% and 150% of the underlying price. These checks remove observations outside the configured analysis range.

Liquidity filtering then applies a minimum open interest of 50 and a maximum relative bid–ask spread of 30%. The relative spread is calculated as:

$$Relative\ Bid - Ask\ Spread = \frac{Ask\ Price - Bid\ Price}{Mid\ Price}$$

If no observations satisfy the strict liquidity conditions, the system retains those passing the initial filters and records that a relaxed liquidity fallback was used. Further liquidity assessment takes place during feature engineering.

```
base_filter = (
    option_chain_prepared["strike_to_spot"].between(
        MIN_STRIKE_TO_SPOT, MAX_STRIKE_TO_SPOT
    )
    & option_chain_prepared["valid_iv"]
    & option_chain_prepared["mid_price"].gt(0)
)
strict_filter = base_filter & option_chain_prepared["liquidity_pass"]
option_universe = option_chain_prepared.loc[strict_filter].copy()
filter_mode = "strict_liquidity"
```

4.3 Underlying Trend and Momentum

Historical underlying prices are used to calculate indicators describing recent price movements. These include 20-day, 50-day and 200-day simple moving averages, the Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD). [3]

A trend score combines four comparisons:

1. Current closing price against the 20-day moving average.
2. Current closing price against the 50-day moving average.
3. The 20-day moving average against the 50-day moving average.
4. MACD against its signal line.

Each comparison contributes +1 when the first value is greater than or equal to the second, and −1 otherwise. The combined score is converted to a scale from 0 to 100:

$$Trend\ Score = \frac{Combined\ Score + 4}{8}\ x\ 100$$

A score above 60 indicates an observed bullish direction, while a score below 40 indicates an observed bearish direction. Scores between these thresholds are treated as neutral. These features are calculated independently of the user's outlook. The reasoning stage can therefore assess whether the user's bullish or bearish view agrees with the observed market trend.

```
components = pd.DataFrame({
    "price_vs_sma20": np.where(close >= result["sma_20"], 1, -1),
    "price_vs_sma50": np.where(close >= result["sma_50"], 1, -1),
    "sma20_vs_sma50": np.where(result["sma_20"] >= result["sma_50"], 1, -1),
    "macd_vs_signal": np.where(result["macd"] >= result["macd_signal"], 1, -1),
}, index=result.index)
insufficient = result[["sma_20", "sma_50", "macd_signal"]].isna().any(axis=1)
result["trend_raw_score"] = components.sum(axis=1).where(~insufficient)
result["trend_score"] = ((result["trend_raw_score"] + 4) / 8 * 100).clip(0, 100)
result["trend_label"] = pd.cut(
    result["trend_score"], bins=[-np.inf, 25, 40, 60, 75, np.inf],
    labels=["strong_bearish", "bearish", "neutral", "bullish", "strong_bullish"],
    include_lowest=True
).astype("object")
return result
```

4.4 Volatility and Expected Price Movement

Historical volatility is calculated from the standard deviation of daily logarithmic returns over 20-day and 60-day windows. The values are annualized using 252 trading days:

$$Historial\ Volatility = Std\ (Daily\ Log\ Returns)\ x\ \sqrt{252}$$

For each expiry, the system estimates at-the-money implied volatility using the median implied volatility of observations with strikes within 2% of the underlying price. If none falls within this range, the nearest available call and put observations are used. The ratio of implied volatility to 20-day historical volatility determines the volatility regime.

Implied-to-historical volatility ratio	**Volatility Regime**
Below 0.80	Low Implied Volatility relative to Historical Volatility
0.80 to 1.20	Balanced Implied and Historical Volatility
Above 1.20 to 1.50	Elevated Implied Volatility
Above 1.50	High Implied Volatility

```python
def volatility_regime(iv_to_hv_ratio: float) -> str:
    if not np.isfinite(iv_to_hv_ratio):
        return "unknown"
    if iv_to_hv_ratio < 0.80:
        return "low_iv_relative_to_hv"
    if iv_to_hv_ratio <= 1.20:
        return "balanced_iv_hv"
    if iv_to_hv_ratio <= 1.50:
        return "elevated_iv"
    return "high_iv"
```

The estimated one-standard-deviation price movement is calculated as:

$$Expected\ Move = S_0\ x\ \sigma_{ATM}\ x\ \sqrt{\frac{DTE}{365}}$$

where S_0 is the underlying price, σ_{ATM} is the estimated at-the-money implied volatility and DTE is the number of calendar days to expiry. The expected lower and upper prices are obtained by subtracting and adding this movement to the underlying price, with the lower value bounded at zero. These estimates provide a common basis for assessing potential price movement across expiries.

```python
def build_expiry_features(options: pd.DataFrame, historical_vol_20: float) -> pd.DataFrame:
    rows: list[dict[str, Any]] = []
    for (expiry, dte), group in options.groupby(["expiry", "dte"]):
        atm = choose_atm_contracts(group)
        spot = float(group["spot_price"].median())
        atm_iv = float(atm["impliedVolatility"].median())
        expected_move_pct = atm_iv * math.sqrt(float(dte) / 365)
        expected_move = spot * expected_move_pct
        iv_hv_ratio = atm_iv / historical_vol_20 if historical_vol_20 > 0 else np.nan
```

4.5 Liquidity and Options Features

The system calculates a liquidity score using open interest, trading volume and bid–ask spread tightness. Each measure is converted into a percentile-based score within the same expiry and option type. The combined liquidity score is calculated as:

$$Liquidity\ Score = 0.45\ (Open\ Interest\ Score) + 0.25\ (Volume\ Score) + 0.30\ (Spread\ Tightness\ Score)$$

The resulting score ranges from 0 to 100. Higher scores indicate stronger liquidity relative to the observations in the same comparison group. A minimum score of 40 is required for the subsequent payoff-analysis stage, together with a positive midpoint, valid implied volatility and positive days to expiry.

Additional features describe whether an option is in, at or out of the money, its time-to-expiry category and whether its strike lies within the estimated price range. These features convert numerical observations into categories that support subsequent analysis.

```
peer_groups = result.groupby(["expiry", "option_type"], group_keys=False)
result["open_interest_score"] = peer_groups["openInterest"].transform(
    lambda values: percentile_score(values.fillna(0), True)
)
result["volume_score"] = peer_groups["volume"].transform(
    lambda values: percentile_score(values.fillna(0), True)
)
result["tight_spread_score"] = peer_groups["bid_ask_spread_pct"].transform(
    lambda values: percentile_score(values, False)
)
result["liquidity_score"] = (
    0.45 * result["open_interest_score"]
    + 0.25 * result["volume_score"]
    + 0.30 * result["tight_spread_score"]
).clip(0, 100)
result["liquidity_band"] = pd.cut(
    result["liquidity_score"], bins=[-np.inf, 25, 50, 75, np.inf],
    labels=["poor", "fair", "good", "excellent"], include_lowest=True
).astype("object")
```

4.6 Quality Checks and Prepared Outputs

Before exporting the prepared data, the system checks that sufficient historical observations are available, historical volatility is positive and underlying prices are consistent across datasets. It also verifies that option identifiers are unique, both option types are available, liquidity scores are bounded and expected movements are positive.

At least one options observation must pass the feature-quality requirements. The expiry feature table must also contain unique combinations of expiry and days to expiry. A quality report records the outcome of each check. If any required check fails, the notebook raises an error before exporting its final feature datasets.

```
spot_from_options = float(option_features["spot_price"].median())
spot_from_history = float(latest_underlying["Close"])
quality_checks = {
    "history_has_enough_rows_for_hv60": len(underlying_features) >= 60,
    "historical_volatility_is_positive": np.isfinite(historical_vol_20) and historical_vol_20 > 0,
    "history_and_option_spot_are_consistent": abs(spot_from_options / spot_from_history - 1) <= 0.05,
    "option_contracts_are_unique": not option_features["contractSymbol"].duplicated().any(),
    "both_option_types_are_available": set(option_features["option_type"]) == {"call", "put"},
    "liquidity_scores_are_bounded": bool(option_features["liquidity_score"].between(0, 100).all()),
    "at_least_one_quality_contract": bool(option_features["candidate_quality_pass"].any()),
    "expected_moves_are_positive": bool(expiry_features["expected_move"].gt(0).all()),
    "expiry_rows_are_unique": not expiry_features[["expiry", "dte"]].duplicated().any(),
}
```

The main outputs are summarized below.

Output	Contents
Underlying features	Historical prices, moving averages, momentum, volatility and trend indicators
Options features	Prepare options data, liquidity scores, moneyness and expiry categories
Expiry features	Implied volatility, volatility regime and expected price ranges
Feature snapshot	A structure summary of observer market conditions
Quality report	Results of the validation checks

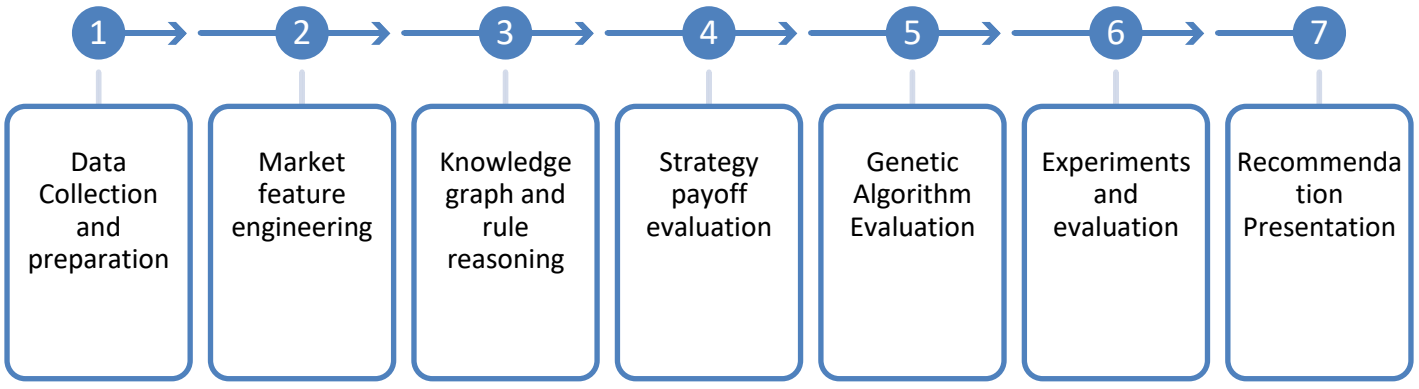
These outputs are saved for use by the knowledge-graph and rule-reasoning stage. Separating data preparation from recommendation logic ensures that the same observed market information can support either a bullish or bearish user outlook.

5. System Architecture and Model Design

The system combines market-data analysis, knowledge representation, rule-based reasoning and genetic optimization to recommend options strategies. Each stage performs a distinct task and produces structured outputs for the next stage. This separation allows the recommendation process to be examined at different levels. Market features describe the observed conditions, reasoning rules assess strategy suitability, and optimization compares feasible alternatives against the defined evaluation objectives.

5.1 System Architecture

The live recommendation pipeline consists of 7 stages:

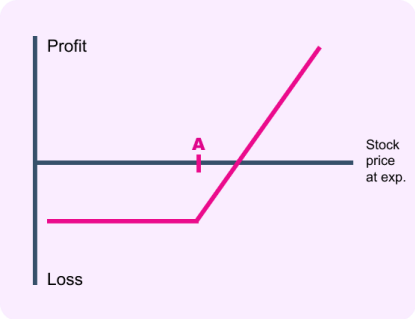


The Streamlit application collects the user's ticker and directional outlook, then executes Notebooks 1–7 in sequence. Each request uses a separate temporary working directory. The pipeline checks for a completion marker after each notebook before continuing to the next stage.

5.2 Supported Options Strategy

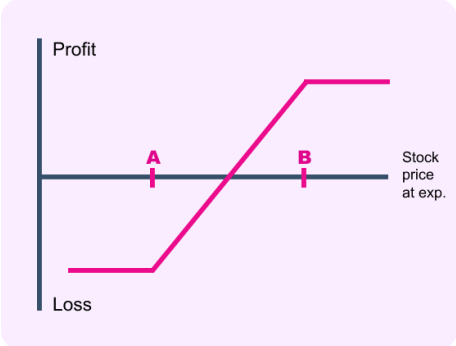
5.2.1 Long Call

A Long Call involves purchasing a call option to express a bullish outlook. [4] The strategy benefits when the underlying price rises sufficiently above the strike price to recover the premium paid. Its maximum loss is limited to the premium, while its theoretical profit potential is unlimited. The breakeven price is the strike price plus the premium paid.



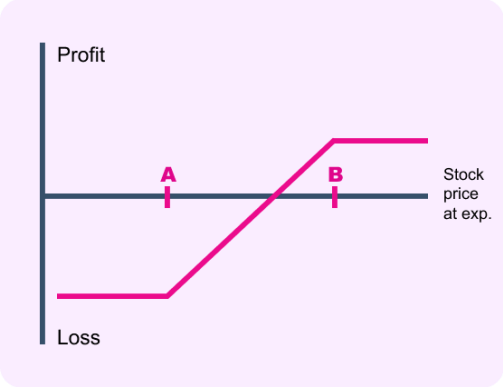
5.2.2 Bull Call Spread

A Bull Call Spread involves buying a lower-strike call and selling a higher-strike call with the same expiry. [5] The strategy requires a net debit and expresses a bullish outlook with limited profit and loss. The maximum loss equals the net premium paid. The maximum profit equals the difference between the strikes minus that premium. Breakeven occurs at the lower strike plus the net premium paid.



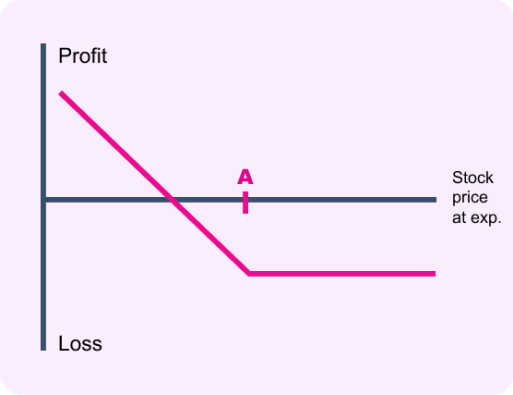
5.2.3 Bull Put Spread

A Bull Put Spread involves selling a higher-strike put and buying a lower-strike put with the same expiry. [6] It receives net credit and expresses a bullish outlook. The maximum profit equals the premium received. The maximum loss equals the difference between the strikes minus that premium. Breakeven occurs at the higher strike minus the net premium received.



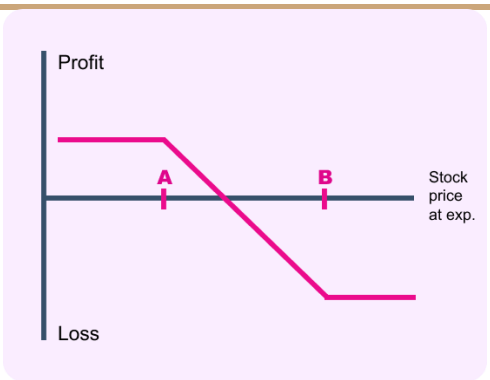
5.2.4 Long Put

A Long Put involves purchasing a put option to express a bearish outlook. [7] The strategy benefits when the underlying price falls sufficiently below the strike price to recover the premium paid. The maximum loss is limited to the premium. The maximum profit occurs if the price falls to zero and equals the strike price minus the premium paid. Breakeven occurs at the strike price minus the premium.



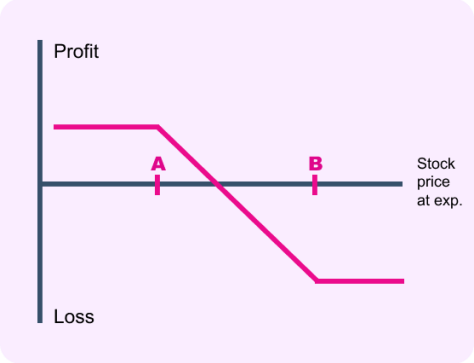
5.2.5 Bear Put Spread

A Bear Put Spread involves buying a higher-strike put and selling a lower-strike put with the same expiry. [8] It requires a net debit and expresses a bearish outlook with limited profit and loss. The maximum loss equals the net premium paid. The maximum profit equals the difference between the strikes minus that premium. Breakeven occurs at the higher strike minus the net premium paid.



5.2.6 Bear Call Spread

A Bear Call Spread involves selling a lower-strike call and buying a higher-strike call with the same expiry. [9] It receives a net credit and expresses a bearish outlook. The maximum profit equals the premium received. The maximum loss equals the difference between the strikes minus that premium. Breakeven occurs at the lower strike plus the net premium received.



5.2.7 Overview of each strategy

The 6 strategies are summarized below.

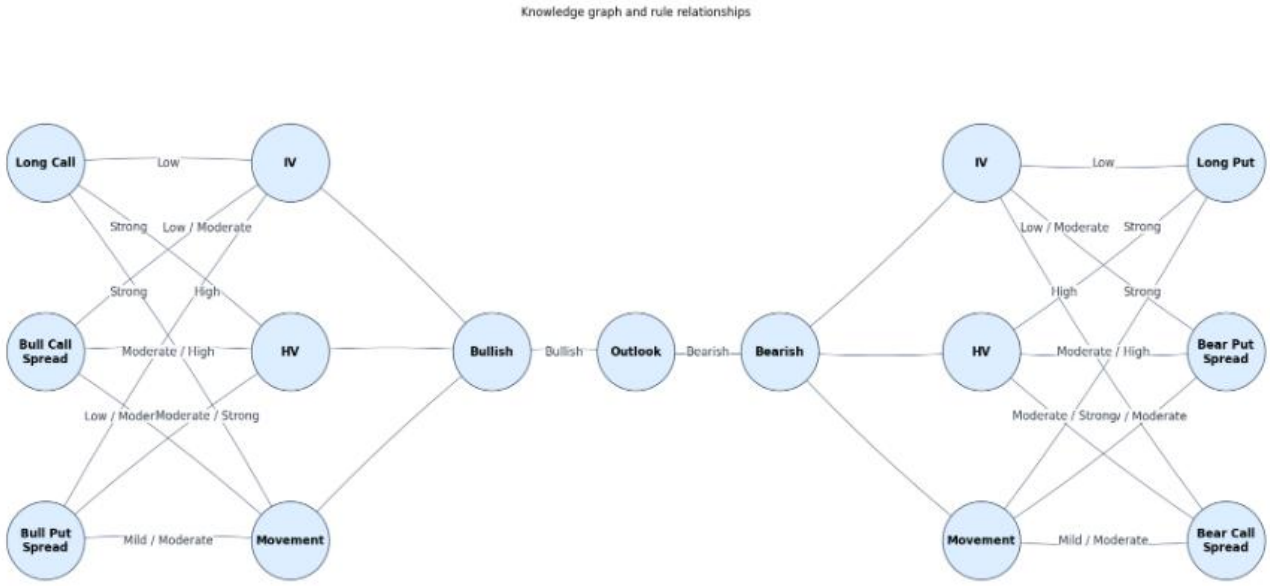
	strategy_id	name	direction	premium_style	leg_count	max_risk	max_profit	preferred_volatility	description
0	bull_call_spread	Bull Call Spread	bullish	debit	2	limited	limited	low_or_balanced	Buy a lower-strike call and sell a higher-strike call.
1	bull_put_spread	Bull Put Spread	bullish	credit	2	limited	limited	elevated_or_high	Sell a higher-strike put and buy a lower-strike put.
2	long_call	Long Call	bullish	debit	1	limited	unlimited	low_or_balanced	Buy a call to express a bullish view with limited premium risk.
3	bear_put_spread	Bear Put Spread	bearish	debit	2	limited	limited	low_or_balanced	Buy a higher-strike put and sell a lower-strike put.
4	bear_call_spread	Bear Call Spread	bearish	credit	2	limited	limited	elevated_or_high	Sell a lower-strike call and buy a higher-strike call.
5	long_put	Long Put	bearish	debit	1	limited	substantial	low_or_balanced	Buy a put to express a bearish view with limited premium risk.

5.3 Knowledge Graph and Rule-Based Strategy Reasoning

The knowledge graph and rule-based reasoning stage translates the user's directional outlook and observed market conditions into a ranked list of options strategies. This process is implemented in the "03_Knowledge_Graph_and_Rule_Reasoning.ipynb" notebook.

5.3.1 Knowledge Graph Representation

Knowledge graphs represent entities and their relationships using a graph-based structure [10]. The knowledge graph begins with the Outlook node, which branches into Bullish and Bearish directions. Each direction connects to three market characteristics: implied volatility (IV), historical volatility (HV) and price movement.



The diagram associates long-option strategies with low implied volatility and strong directional movement. Debit spreads are associated with low to moderate implied volatility and low to moderate movement, while credit spreads are associated with high implied volatility and mild to moderate movement.

These relationships provide a conceptual overview of strategy suitability. In the implemented scoring process, volatility is assessed through the ratio of implied volatility to historical volatility. The individual IV, HV and movement labels shown in the diagram are not applied as separate scoring rules.

5.3.2 Direction and Volatility Rules

The user's outlook determines which three strategies are evaluated. A bullish outlook includes Long Call, Bull Call Spread and Bull Put Spread, while a bearish outlook includes Long Put, Bear Put Spread and Bear Call Spread. The system selects a reference expiry based on the availability of quality observations and median liquidity. Its volatility regime determines the base score for each strategy.

Strategy	Low or balanced IV base score	Elevated or high IV base score
Long Call	85	45
Bull Call Spread	95	75
Bull Put Spread	55	100
Long Put	85	45
Bear Put Spread	95	75
Bear Call Spread	55	100

The scores favor debit strategies when implied volatility is low or balanced relative to historical volatility. Credit strategies receive higher scores when implied volatility is elevated or high. These values are predefined design choices rather than parameters learned from historical trading results. In the code, only elevated_iv and high_iv activate the high-IV branch; other regimes, including an unknown regime, follow the non-high-IV branch.

5.3.3 Market Condition Adjustments

The base scores are adjusted according to trend alignment, liquidity and data availability. Trend alignment compares the user's outlook with the observed market direction. An aligned outlook receives 10 additional points, a conflicting outlook receives a deduction of 20 points, and a neutral market direction produces no adjustment. Liquidity is assessed using the median liquidity score of reference expiry. Higher liquidity produces a positive adjustment, while poor liquidity reduces the score.

Factor	Condition	Score Adjustment
Trend Alignment	Aligned	+10
	Neutral	0
	Conflicting	-20
Median Liquidity	75 or above	+10
	50 to below 75	+5
	25 to below 50	0
	Below 25	-15
Data Availability	Fewer than 4 quality observations for 2 leg strategy	-20

The final reasoning score is calculated as:

$$Reasoning\ Score = clip(Base\ Score + Trend\ Adjustment + Liquidity\ Adjustment + Availability\ Adjustment, 0, 120)$$

The clipping operation ensures that the score remains between 0 and 120.

5.3.4 Strategy Ranking and Explanation

The system ranks strategies by their final reasoning scores in descending order. When scores are equal, strategy name determines the order. With the default setting of TOP_STRATEGY_COUNT = 3, all three strategies matching the user's outlook proceed to payoff evaluation.

Each ranked strategy includes an explanation describing its directional suitability, volatility preference, trend alignment and liquidity adjustment. Any availability penalty is also recorded. A reasoning trace preserves the rules applied and their contributions to the score.

The recommendation results are added to the RDF knowledge graph and exported alongside the ranked strategy table and reasoning trace. These outputs make the reasoning process inspectable and provide the rule-alignment information used in the subsequent optimization stage.

	rank	name	reasoning_score	selected_expiry	volatility_regime	explanation
0	1	Bull Call Spread	110.000	2026-10-16	low_iv_relative_to_hv	Direction rule: Bull Call Spread supports the user's bullish outlook. | low_iv_relative_to_hv favours debit structures. | Trend alignment is aligned (+10 po...
1	2	Long Call	100.000	2026-10-16	low_iv_relative_to_hv	Direction rule: Long Call supports the user's bullish outlook. | low_iv_relative_to_hv favours debit structures. | Trend alignment is aligned (+10 points). ...
2	3	Bull Put Spread	70.000	2026-10-16	low_iv_relative_to_hv	Direction rule: Bull Put Spread supports the user's bullish outlook. | low_iv_relative_to_hv is less favourable for this credit structure. | Trend alignment...

6. Strategy Optimization and Evaluation

The strategy optimisation stage evaluates the alternatives produced by the reasoning process and identifies a recommendation using a combination of return, risk and liquidity objectives. It is implemented across the "04_Option_Strategy_Payoff_Engine.ipynb", "05_Genetic_Algorithm_Strategy_Optimisation.ipynb" and "06_Experiments_and_Evaluation.ipynb".

The process begins with payoff calculation and simulated outcome estimation. Transaction costs are then incorporated before a genetic algorithm search for a strategy configuration with a high fitness score. Evaluation procedures compare the recommendation with alternative selection methods and examine its sensitivity to different assumptions.

6.1 Strategy Payoff Evaluation

The payoff engine calculates the expiration profit or loss of each valid strategy configuration. It uses the intrinsic value of its component options and the initial premium paid or received.

Call and put intrinsic values are calculated as:

$$Call\ Intrinsic\ Value = \max(S_T - K, 0)$$
$$Put\ Intrinsic\ Value = \max(K - S_T, 0)$$

where S_T is the underlying price at expiry and K is the strike price.

For a spread, the intrinsic value of the sold option is deducted from that of the purchased option. The initial cash flow is then added, with a negative value representing a debit and a positive value representing a credit. The result is multiplied by 100 to obtain the profit or loss for the standard strategy unit used by the system.

The engine also calculates maximum profit, maximum loss and breakeven. Configurations with invalid strategy structures, unsuitable net premiums or non-positive maximum-loss values are rejected before further evaluation.

```
def analytical_metrics(strategy_id: str, long_strike: float, short_strike: float | None, premium: float) -> dict[str, float]:
    if strategy_id == "long_call":
        return {"max_profit": np.inf, "max_loss": premium * CONTRACT_MULTIPLIER, "breakeven": long_strike + premium}
    if strategy_id == "long_put":
        return {"max_profit": max(0, long_strike - premium) * CONTRACT_MULTIPLIER, "max_loss": premium * CONTRACT_MULTIPLIER, "breakeven": long_strike -
    width = abs(float(short_strike) - long_strike)
    if strategy_id == "bull_call_spread":
        return {"max_profit": (width - premium) * CONTRACT_MULTIPLIER, "max_loss": premium * CONTRACT_MULTIPLIER, "breakeven": long_strike + premium}
    if strategy_id == "bear_put_spread":
        return {"max_profit": (width - premium) * CONTRACT_MULTIPLIER, "max_loss": premium * CONTRACT_MULTIPLIER, "breakeven": long_strike - premium}
    if strategy_id == "bull_put_spread":
        return {"max_profit": premium * CONTRACT_MULTIPLIER, "max_loss": (width - premium) * CONTRACT_MULTIPLIER, "breakeven": float(short_strike) - pre
    if strategy_id == "bear_call_spread":
        return {"max_profit": premium * CONTRACT_MULTIPLIER, "max_loss": (width - premium) * CONTRACT_MULTIPLIER, "breakeven": float(short_strike) + pre
    raise ValueError(f"Unsupported strategy: {strategy_id}")
```

6.2 Monte Carlo Outcome Estimation

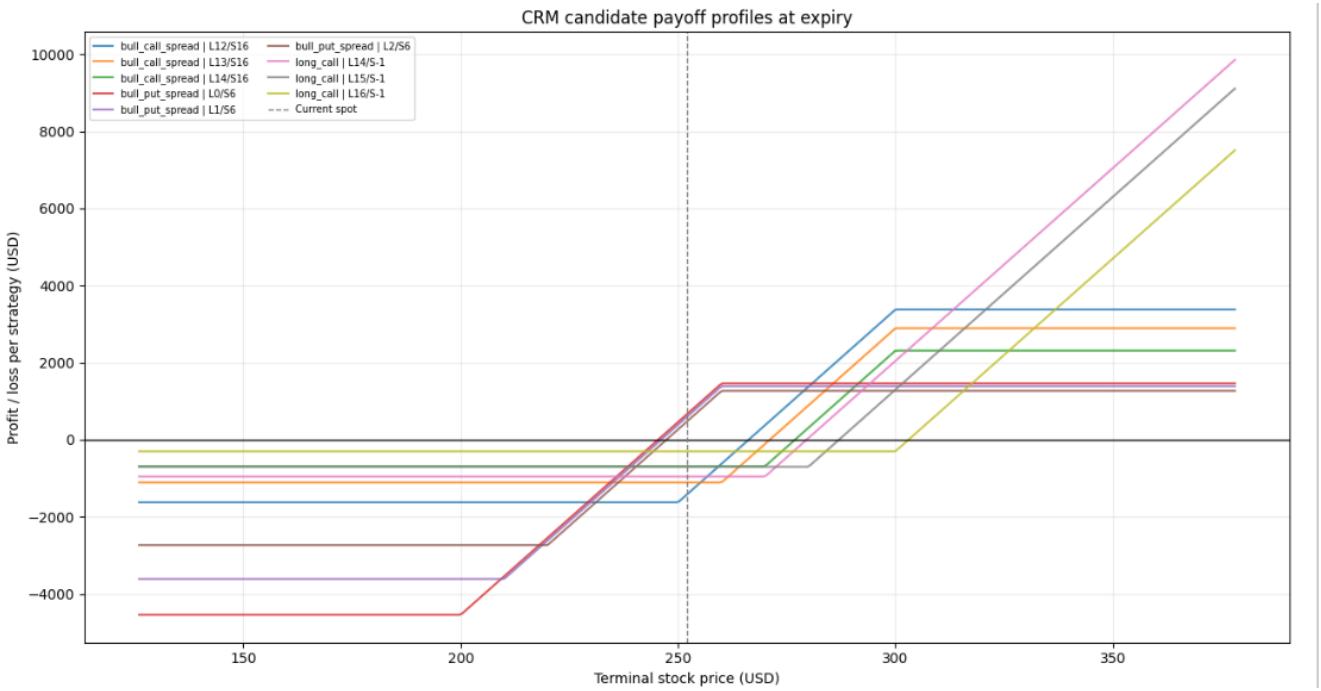
Monte Carlo simulation evaluates financial outcomes across randomly sampled scenarios [11]. The system uses Monte Carlo simulation to estimate strategy outcomes under a range of possible underlying prices at expiry. The default configuration generates 20,000 simulated terminal prices using a fixed random seed of 42. Terminal prices follow the model:

$$S_T = S_0 \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)T + \sigma\sqrt{T}Z\right]$$

where:

- S_0 is the current underlying price
- μ is the annualised drift estimated from recent momentum
- σ is the at-the-money implied volatility
- T is the time to expiry in years
- Z is a standard normal random variable

The drift is derived from the observed 20-day return and restricted to between −25% and +25% annually. Strategies sharing an expiry are evaluated using the same simulated terminal prices, providing a consistent basis for comparison. Diagram below shows different outcome of different strategy at expiry using “CRM” and bullish outlook as example.



The resulting profit-and-loss values are used to calculate expected profit, probability of profit, median profit and the 5th and 95th percentile outcomes.

$$Expected\ Profit = \frac{1}{N}\sum_{i=1}^{N} Profit_i$$

$$Probability\ of\ Profit = \frac{Number\ of\ Simulations\ with\ Positive\ Profit}{N}$$

6.3 Transaction Cost Adjustment

Notebook 5 incorporates estimated commissions, regulatory fees and slippage into the strategy evaluation. The configured assumptions are summarised below.

Cost component	Value
Commissions	$0.65
Regulatory Fees	$0.03
Slippage	1% of premiums

The estimated costs are deducted from expected profit and added to maximum loss:

$$Adjusted\ Expected\ Profit = Expected\ Profit - Estimated\ Transaction\ Cost$$

$$Adjusted\ Maximum\ Loss = Maximum\ Loss + Estimated\ Transaction\ Cost$$

The net expected return on risk is then calculated as:

$$Net\ Expected\ Return\ on\ Risk = \frac{Adjusted\ Expected\ Profit}{Adjusted\ Maximum\ Loss}$$

```python
def add_cost_adjustments(frame: pd.DataFrame, cost_multiplier: float = 1.0) -> pd.DataFrame:
    scored = frame.copy()
    scored["leg_count"] = np.where(scored["short_strike_index"].lt(0), 1, 2)
    scored["gross_premium_per_share"] = scored["long_entry_price"].abs() + scored["short_entry_price"].fillna(0).abs()
    fixed_cost = scored["leg_count"] * 2 * (
        COMMISSION_PER_CONTRACT_PER_SIDE + REGULATORY_FEE_PER_CONTRACT_PER_SIDE
    )
    slippage_cost = scored["gross_premium_per_share"] * 100 * SLIPPAGE_RATE_ON_GROSS_PREMIUM
    scored["estimated_transaction_cost"] = cost_multiplier * (fixed_cost + slippage_cost)
    scored["adjusted_expected_profit"] = scored["expected_profit"] - scored["estimated_transaction_cost"]
    scored["adjusted_max_loss"] = scored["max_loss"] + scored["estimated_transaction_cost"]
    scored["net_expected_return_on_risk"] = scored["adjusted_expected_profit"] / scored["adjusted_max_loss"].clip(lower=1e-9)
    scored["adjusted_p05_profit"] = scored["p05_profit"] - scored["estimated_transaction_cost"]
    scored["downside_loss_ratio"] = (-scored["adjusted_p05_profit"].clip(upper=0)) / scored["adjusted_max_loss"].clip(lower=1e-9)
    return scored
```

6.4 Multi-Objective Fitness Function

A weighted-sum approach combines multiple evaluation criteria into a single objective, with weights expressing their relative importance [12]. The optimization process uses a weighted fitness function to balance 6 objectives.

Fitness Component	Weight	Purpose
Net Expectancy	30%	Reward higher cost-adjusted expected return on risk
Probability of Profit	20%	Reward higher simulated likelihood of positive profit
Downside Protection	15%	Favor smaller 5th percentile loss relative to maximum risk
Capital Efficiency	10%	Favor lower adjusted maximum loss
Liquidity	15%	Reward stronger liquidity
Reasoning Alignment	10%	Preserver the influence of strategy reasoning score

The component values are normalised before combination. The system uses the 5th and 95th percentiles as scaling limits and clips values outside these limits. This reduces the influence of extreme observations. Components for which lower values are preferred are reversed after scaling.

The final fitness score is calculated as:

$$Fitness\ Score = 100 \sum_{j=1}^{6} w_j c_j$$

where w_j is the weight of component j, c_j is its normalised value, and the weights sum to one. The score measures relative suitability within the prepared candidate universe. A high score indicates a favourable balance under the configured objectives but not necessary imply positive expected profit.

```python
def build_scored_table(frame: pd.DataFrame, cost_multiplier: float = 1.0) -> pd.DataFrame:
    scored = add_cost_adjustments(frame, cost_multiplier=cost_multiplier)
    scored["component_net_expectancy"] = robust_unit_scale(scored["net_expected_return_on_risk"], True)
    scored["component_probability_of_profit"] = robust_unit_scale(scored["probability_of_profit"], True)
    scored["component_downside_protection"] = robust_unit_scale(scored["downside_loss_ratio"], False)
    scored["component_capital_efficiency"] = robust_unit_scale(scored["adjusted_max_loss"], False)
    scored["component_liquidity"] = robust_unit_scale(scored["liquidity_score"], True)
    scored["component_reasoning_alignment"] = robust_unit_scale(scored["reasoning_score"], True)
    scored["fitness_score"] = 100 * sum(
        FITNESS_WEIGHTS[name] * scored[f"component_{name}"] for name in FITNESS_WEIGHTS
    )
    scored["fitness_rank"] = scored["fitness_score"].rank(method="first", ascending=False).astype(int)
    return scored.sort_values(["fitness_rank", "strategy_rank", "payoff_engine_rank"]).reset_index(drop=True)
```

6.5 Genetic Algorithm Optimization

Genetic algorithms use evolutionary mechanisms such as selection, crossover and mutation to search for candidate solutions [13]. The genetic algorithm searches the feasible strategy configurations generated by the payoff engine. Each configuration is represented by an integer chromosome containing the strategy and its associated parameter indices.

The initial population is sampled from the feasible universe. Tournament selection favours individuals with higher fitness. Uniform crossover combines information from two parents, while mutation

introduces variation. Elitism preserves the strongest individuals between generations, and random immigrants introduce additional diversity.

Genetic Algorithm Parameter	Value
Population size	Up to 80
Generations	60
Tournament size	3
Crossover probability	0.85
Mutation Probability	0.18
Elite Fraction	10%
Random immigrant fraction	5%

```
def run_genetic_algorithm(seed: int) -> tuple[dict[str, Any], pd.DataFrame]:
    rng = np.random.default_rng(seed)
    population_size = max(12, min(POPULATION_SIZE, max(12, len(feasible_chromosomes) * 2)))
    initial_indices = rng.integers(0, len(feasible_chromosomes), size=population_size)
    population = feasible_chromosomes[initial_indices].copy()
    elite_count = max(1, int(round(population_size * ELITE_FRACTION)))
    immigrant_count = max(1, int(round(population_size * RANDOM_IMMIGRANT_FRACTION)))
    history_rows: list[dict[str, Any]] = []
```

The best individual in the final population becomes the primary recommendation. The notebook records the best, mean and median fitness at every generation, together with population diversity.

```
    final_scores = np.array([fitness_by_chromosome[chromosome_tuple(item)] for item in population])
    best_chromosome = population[int(np.argmax(final_scores))]
    best_key = chromosome_tuple(best_chromosome)
    result = {
        "seed": seed,
        "best_chromosome": best_key,
        "best_fitness": float(fitness_by_chromosome[best_key]),
        "candidate_row": int(row_by_chromosome[best_key]),
    }
    return result, pd.DataFrame(history_rows)
```

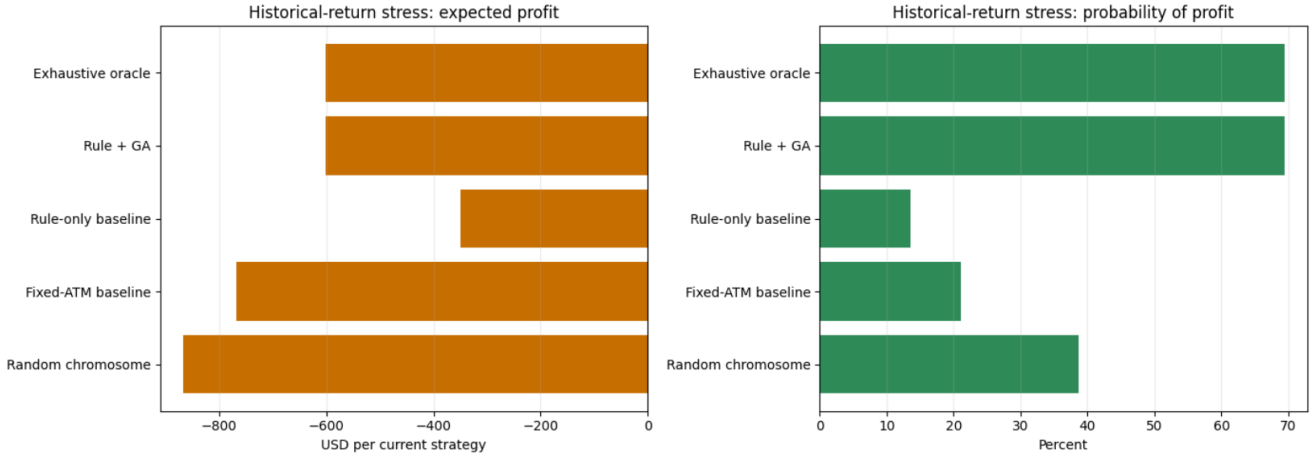
6.6 Evaluation

The recommendation (Rule + GA) is compared with 4 alternative methods.

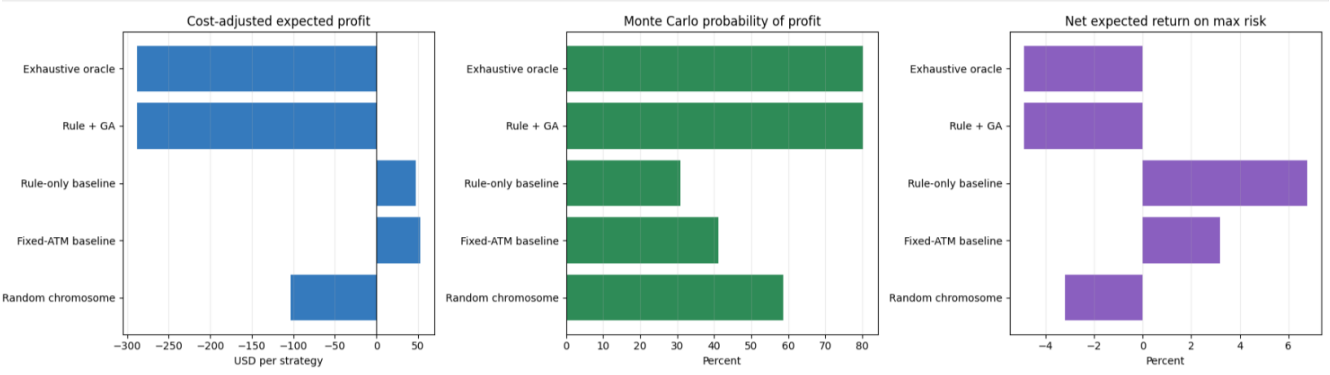
Method	Role
Random Chromosome	Provides a baseline using randomly generated feasible configurations
Fixed-ATM baseline	Select a configuration near the ATM strike within the highest ranked strategy family
Rule-only baseline	Uses the highest ranked family and its payoff engine ranking
Rule + GA	Uses the primary genetic algorithm recommendation

Exhaustive oracle	Identifies the highest fitness in the complete prepared candidate universe

The Rule + GA achieves the joint-highest probability of profit in both evaluations, at approximately 80% under Monte Carlo simulation and 69% under historical-return stress testing. These results exceed those of the rule-only, fixed-ATM, and random chromosome baselines. Its strong performance on this metric indicates that the selected configuration generates positive outcomes across a larger proportion of the evaluated scenarios.



Despite its high probability of profit, the Rule + GA records a cost-adjusted expected loss of approximately US$290 under Monte Carlo simulation and an expected loss of approximately US$600 under historical stress. These results demonstrate that the frequency of profitable outcomes alone does not determine overall profitability: less frequent losses can be sufficiently large to outweigh more frequent gains. The negative expected profit cannot be attributed to computational costs.

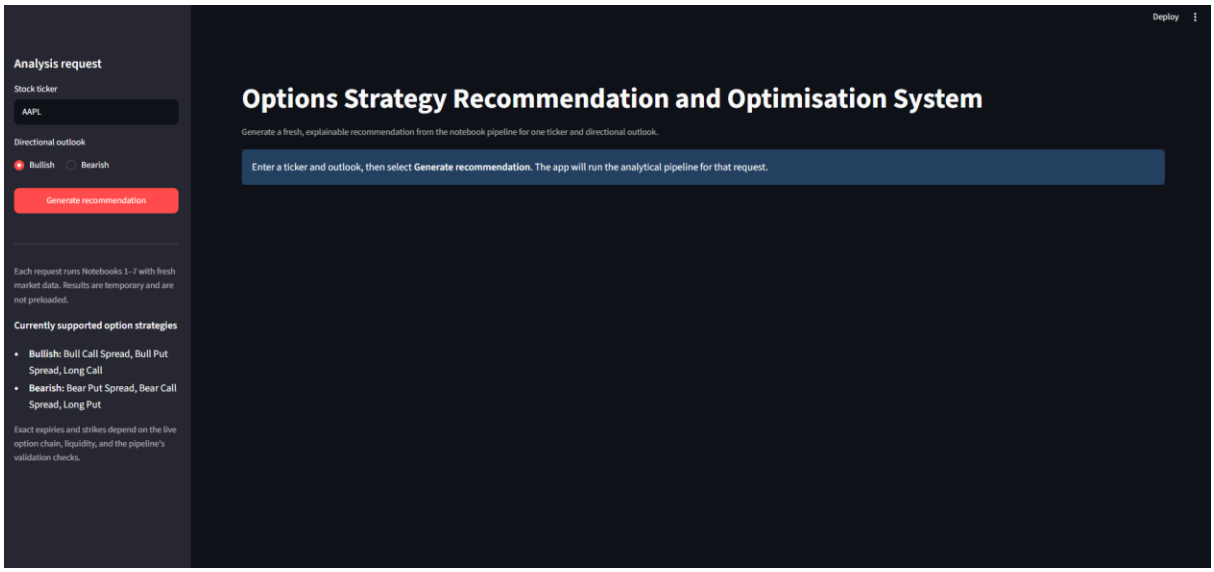


These findings reinforce the role of the system as a decision-support tool. A high probability of profit or a highest-ranked fitness score should be assessed alongside expected profit, potential loss magnitude and scenario sensitivity. They also support introducing a no-trade condition when the available recommendations fail minimum profitability or risk requirements, with that condition evaluated on new data before adoption.

7. End-to-End Recommendation Application

The recommendation application integrates the analytical pipeline into an interactive interface using Streamlit. [14] It allows users to submit a stock ticker and directional outlook, then view the recommended strategy together with its explanation, payoff profile and evaluation evidence. The recommendation output is assembled in the “07_End_to_End_System_Demo.ipynb” notebook. User interface and notebook execution are implemented in application/app.py and application/pipeline.py.

The application’s startup interface is shown below:



7.1 User Input and Pipeline Execution

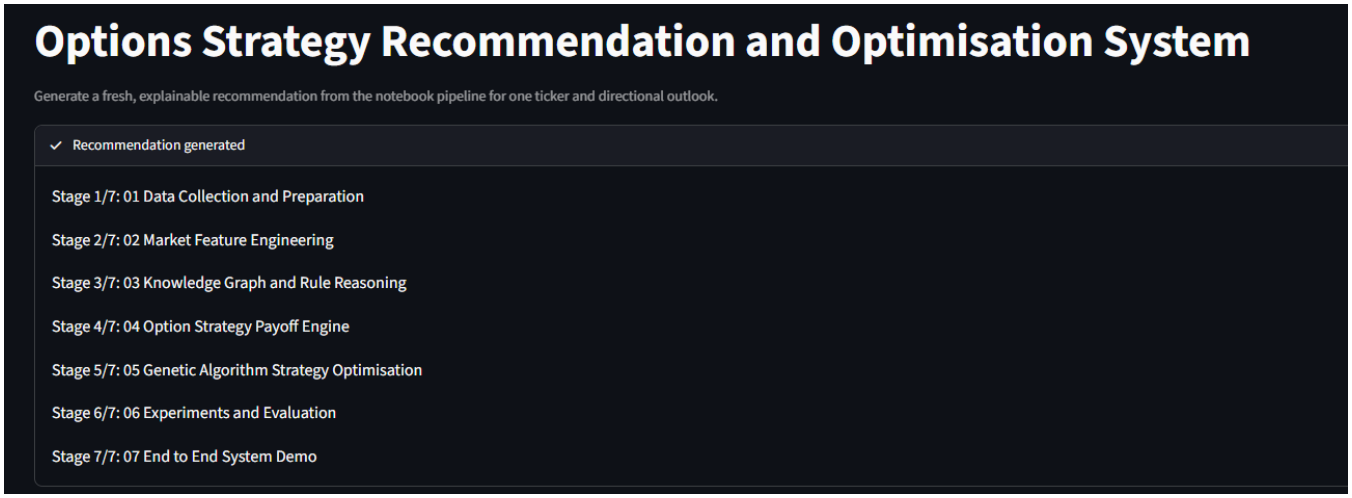
The application is developed using Streamlit. Its sidebar contains a stock-ticker input, a choice between Bullish and Bearish, and a Generate recommendation button. It also lists the supported strategies for each direction. When the user submits a request, the application standardises and validates the inputs before running Notebooks 1–7 sequentially through Papermill. The ticker is passed to each notebook, while the directional outlook is introduced from the reasoning stage onward.

```
with st.sidebar:
    st.header("Analysis request")
    with st.form("analysis-request", border=False):
        ticker_input = st.text_input("Stock ticker", value="AAPL", max_chars=15)
        outlook_input = st.radio(
            "Directional outlook",
            options=["Bullish", "Bearish"],
            horizontal=True,
        )
        generate = st.form_submit_button(
            "Generate recommendation",
            type="primary",
            use_container_width=True,
        )
```

7.2 Recommendation Assembly and Validation

Notebook 7 combines the outputs of the feature-engineering, reasoning, optimisation and evaluation stages. It loads the market summary, ranked strategies, primary genetic algorithm recommendation, method comparisons and quality reports. The notebook verifies that the upstream records match the requested ticker and outlook and the recommendation identified by the genetic algorithm.

The selected strategy is then combined with its market context, payoff information and explanations. The notebook checks the consistency of the strategy's cash flows and confirms that the required upstream quality checks have passed. The validated output is saved as a structured recommendation file, accompanied by tables containing the payoff curve, scenarios, alternatives and pipeline status. The application reads these files to populate the interface.

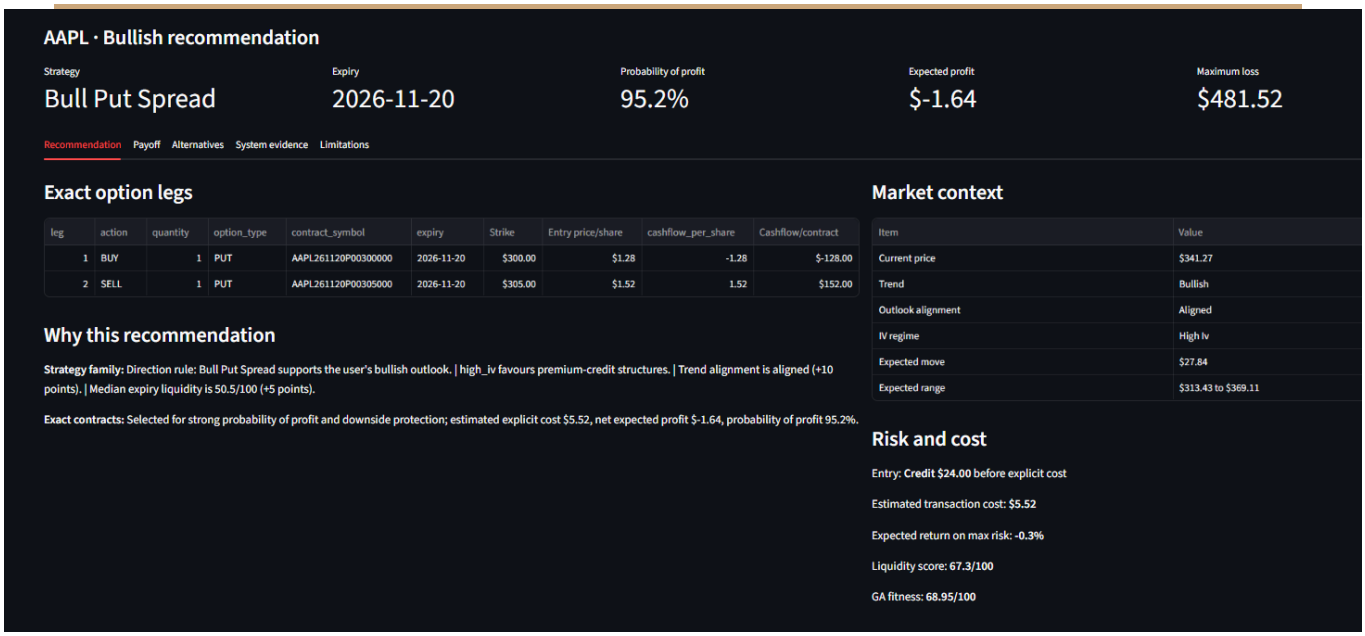


7.3 Recommendation and Market Context

The main interface displays 5 summary measures: recommended strategy, expiry, probability of profit, expected profit and maximum loss. Expected profit and maximum loss include the estimated explicit transaction costs, while probability of profit retains the pre-cost Monte Carlo estimation.

The Recommendation tab provides the strategy details and two explanations. The knowledge explanation describes the reasoning behind the strategy-family assessment. The optimisation explanation identifies the main fitness contributions supporting the selected configuration.

A market-context table presents the underlying price, observed trend, outlook alignment, volatility regime and expected price range. The interface also displays estimated transaction costs, expected return on maximum risk, liquidity score and fitness score.



7.4 Payoff Visualisation and Alternatives

The Payoff tab displays the strategy's cost-adjusted profit or loss across a range of underlying prices at expiry. The curve combines the expiration payoff, initial premium cash flow and estimated explicit transaction costs.

A scenario table shows the payoff at selected price levels, including the expected lower price, current price, breakeven estimates and expected upper price. The cost-adjusted breakeven is approximated using the point on the generated price grid with the smallest absolute profit or loss.



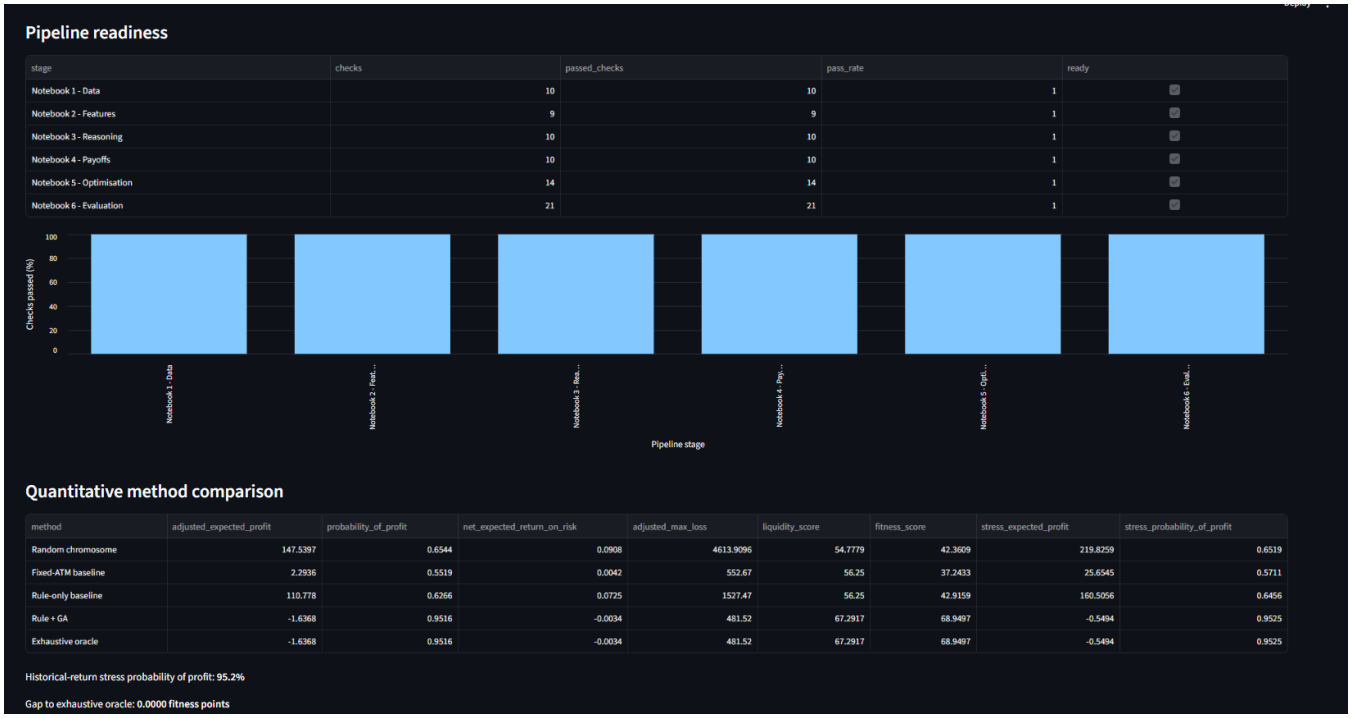
The Alternatives tab presents up to five other highly ranked configurations. These may include different configurations of the same strategy family. The comparison shows expected profit, probability of profit, maximum loss, liquidity and fitness.



fitness_rank	strategy_name	expiry	long_strike	short_strike	adjusted_expected_profit	probability_of_profit	adjusted_max_loss	liquidity_score	fitness_score
2	Bull Put Spread	2026-11-20	295	305	20.1201	0.9528	947.18	63.75	68.4277
3	Bull Put Spread	2026-11-20	295	300	3.607	0.969	483.81	63.75	67.9972
4	Bull Put Spread	2026-11-20	270	295	37.2777	0.9813	2445.86	62.0833	66.0246
5	Bull Put Spread	2026-11-20	270	300	53.4047	0.9709	2917.15	62.0833	65.8038
6	Bull Put Spread	2026-11-20	270	305	69.9179	0.9554	3380.52	62.0833	65.5484

7.5 System Evidence and Limitation

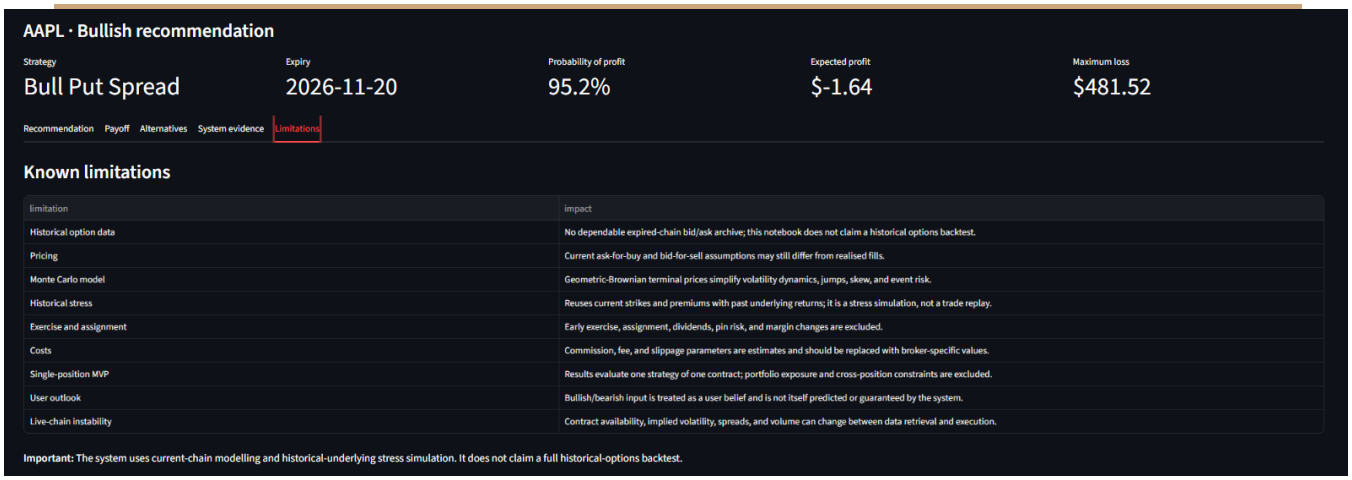
The System evidence tab displays the results of the pipeline quality checks and quantitative method comparisons. Users can inspect the proportion of checks passed at each upstream stage and compare the recommendation with the baseline methods.



stage	checks	passed_checks	pass_rate	ready
Notebook 1 - Data	10	10	1	
Notebook 2 - Features	9	9	1	
Notebook 3 - Reasoning	10	10	1	
Notebook 4 - Payoffs	10	10	1	
Notebook 5 - Optimisation	14	14	1	
Notebook 6 - Evaluation	21	21	1	

method	adjusted_expected_profit	probability_of_profit	net_expected_return_on_risk	adjusted_max_loss	liquidity_score	fitness_score	stress_expected_profit	stress_probability_of_profit
Random chromosome	147.5397	0.6544	0.0908	4613.9096	54.7779	42.3609	219.8259	0.6519
Fixed-ATM baseline	2.2936	0.5519	0.0042	552.67	56.25	37.2433	25.6545	0.5711
Rule-only baseline	110.778	0.6266	0.0725	1527.47	56.25	42.9159	160.5056	0.6456
Rule + GA	-1.6368	0.9516	-0.0034	481.52	67.2917	68.9497	-0.5494	0.9525
Exhaustive oracle	-1.6368	0.9516	-0.0034	481.52	67.2917	68.9497	-0.5494	0.9525

The Limitations tab presents the modelling assumptions and known restrictions. These include simplified terminal-price modelling, uncertain execution prices, estimated transaction costs, and the exclusion of early exercise and assignment.



8. System Evaluation through Historical Backtesting

Historical backtesting is implemented in the “08_Historical_Options_Backtesting.ipynb” notebook. It evaluates the system using SPY options data from 2020 to 2022. A chronological trading policy determines whether to enter a recommended position based on the observed market trend, previously settled strategy outcomes and risk limits.

8.1 Backtesting Methodology

The historical dataset covers 2 January 2020 to 30 December 2022. Before generating recommendations, the notebook checks the observations against the New York Stock Exchange calendar. Records dated on non-trading days are excluded, and missing exchange sessions are identified.

Recommendations are generated at the first available observation of each week after an initial 60-observation warm-up period. The final decision date is restricted to at least 61 calendar days before the end of the dataset to allow time for the evaluated positions to expire.

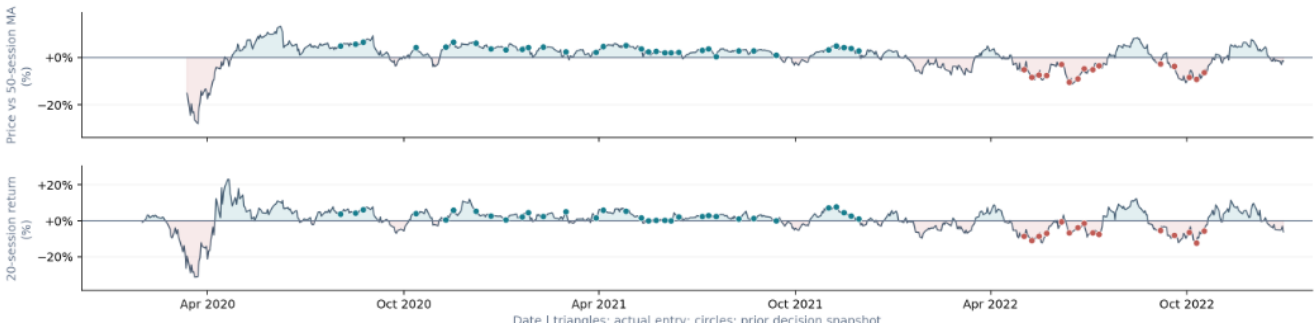
At each decision point, the system prepares historical inputs using only observations available up to that date. Bundled versions of the feature-engineering, reasoning, payoff and genetic algorithm stages then generate recommendations for both directional outlooks. The chronological policy determines whether either recommendation qualifies for entry.

8.2 Chronological Trading Policy

The historical trading policy applies trend, evidence and risk requirements before accepting a recommendation. These requirements are additional to the strategy-ranking rules.

8.2.1 Trend Alignment

The policy identifies market direction using the underlying price relative to its 50-session moving average and its 20-session return. A bullish direction requires the price to be above its 50-session moving average and the 20-session return to be positive. A bearish direction requires the price to be below the moving average and the 20-session return to be negative. Otherwise, the observed direction is classified as neutral. Recommendations matching the observed direction are plotted as below.



8.2.2 Previously Settled Strategy Evidence

A trend-aligned recommendation must also satisfy a historical-evidence requirement. The policy requires at least 5 settled simulated observations of the same strategy, associated with the same directional regime, with expiry dates within the preceding 365 days.

Both the average realised dollar profit and the average return on maximum risk of these observations must be positive. Only outcomes with expiry dates before the current decision date are included. Future outcomes and missing expiry results cannot contribute to the evidence.

8.3 Risk Management and Trade Simulation

A recommendation that satisfies the timing and evidence conditions must also meet the position-risk requirements. The policy uses one option per leg and does not increase position size for inexpensive strategies.

Risk Control	Limit
Maximum loss per selected position	$1000
Combined maximum loss across overlapping positions	$10000
Selected outlooks per decision trade	1

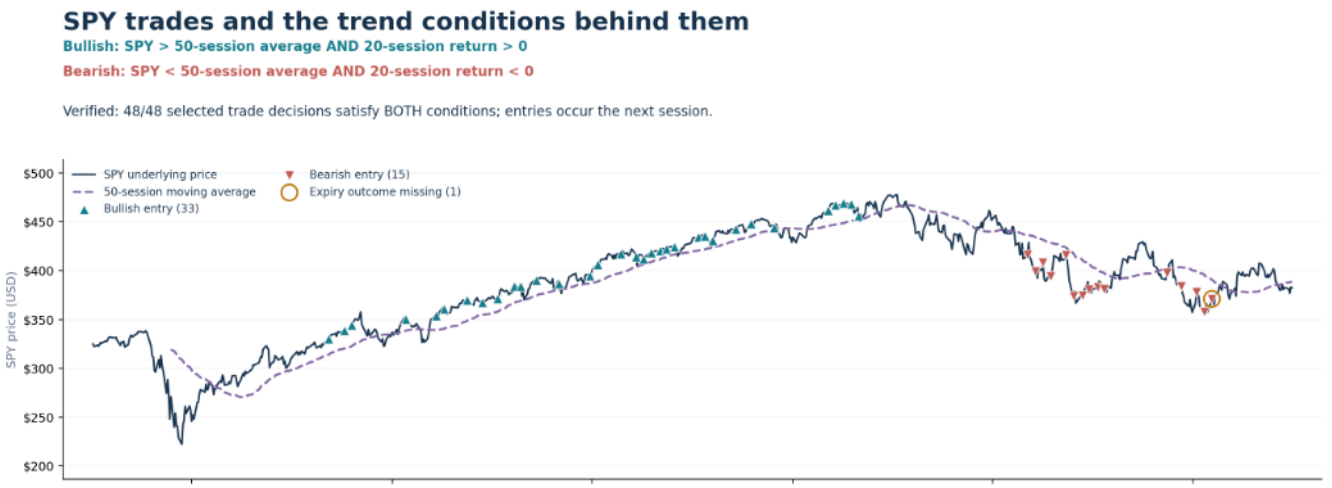
Positions exceeding these limits are skipped. A valid entry with a missing expiry outcome continues to reserve risk capacity until expiry, preventing missing results from creating artificial capacity for subsequent positions. These controls address both individual-position risk and overlapping exposure. If the recommended position fails these checks, the scenario is skipped without substitution.

Finally, the expiration profit or loss is calculated as:

$$Net\ Profit\ or\ Loss = Entry\ Cash\ Flow + Expiration\ Payoff - Trading\ Costs$$

8.4 Overall Backtesting Results

The saved run contains 270 scheduled scenarios. After applying the chronological trading policy, 47 positions were selected and plotted below.



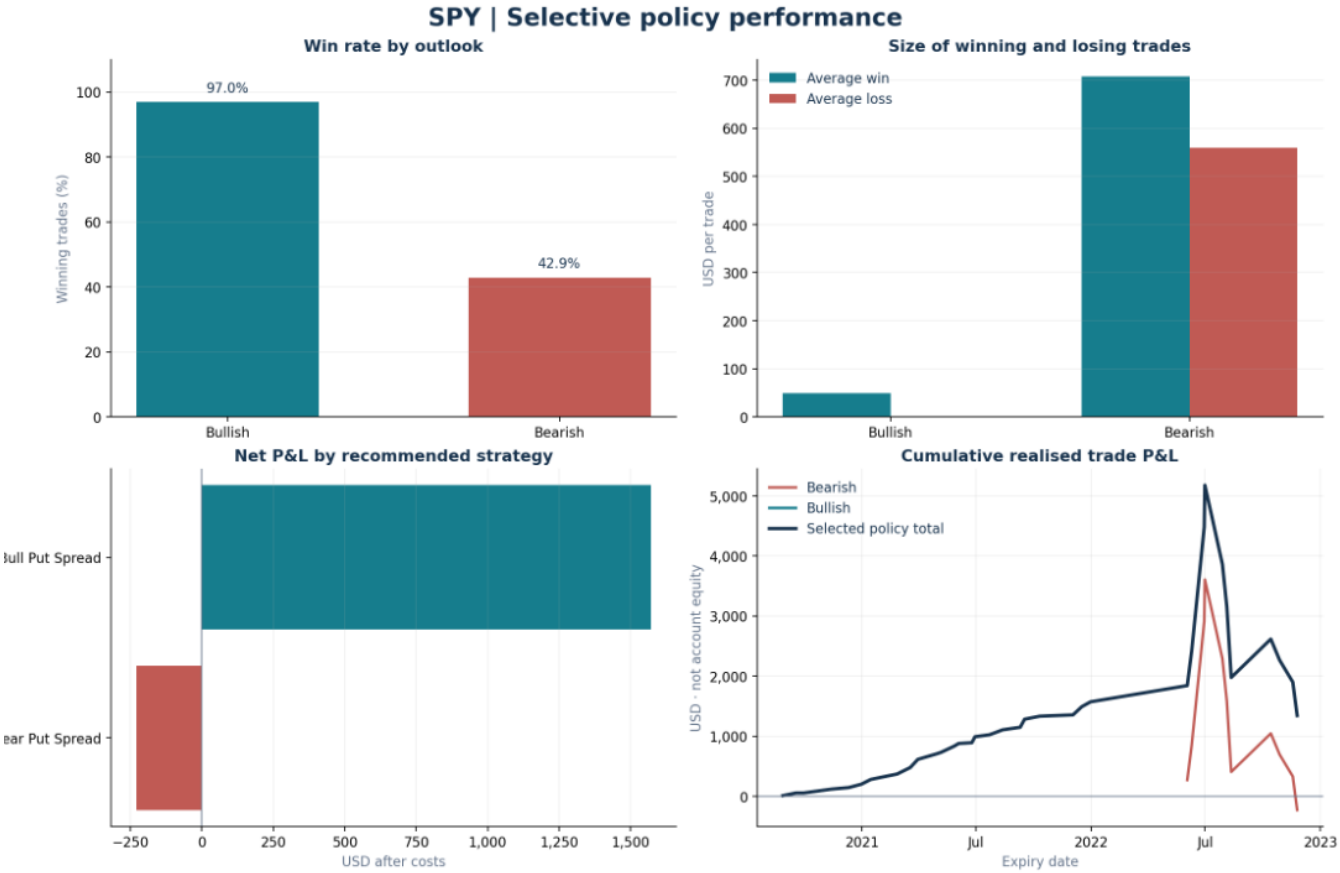
The completed trades generated a net profit of $1,342.08 after costs. The 33 bullish trades achieved a 96.97% win rate and contributed $1,571.12, while the 14 bearish trades achieved a 42.86% win rate and recorded a net loss of $229.04.

Outlook	Trades	Win rate (%)	Average win ($)	Average loss ($)	Net P&L ($)	Profit factor
bearish	14.00	42.86	707.64	559.36	-229.04	0.95
bullish	33.00	96.97	49.11	0.36	1,571.12	4,365.22

47 completed trades · 223 skipped scenarios · 270 scheduled scenarios

The figure highlights contrasting payoff patterns between the two outlooks. Bullish trades achieved frequent, modest gains, with an average win of $49.11 and an average loss of $0.36. Bearish trades had a larger average win of $707.64, but their lower win rate was insufficient to offset average losses of $559.36.

The cumulative realised P&L shows steady initial growth followed by sharp fluctuations during the bearish trading period. This suggests that profitability depended primarily on the bullish component, while bearish positions introduced substantial variability.



The backtesting results illustrate that the system's recommendation is one component of a broader trading decision. It provides structured analysis of options strategies, while trade timing determines when to act and risk management controls the exposure accepted. The reported performance reflects these components operating together. It does not separately establish how much the recommendation engine, timing conditions or risk limits contributed to the outcome. Nevertheless, the variation in results and the size of losing trades show why recommendations should be interpreted alongside market conditions and risk constraints.

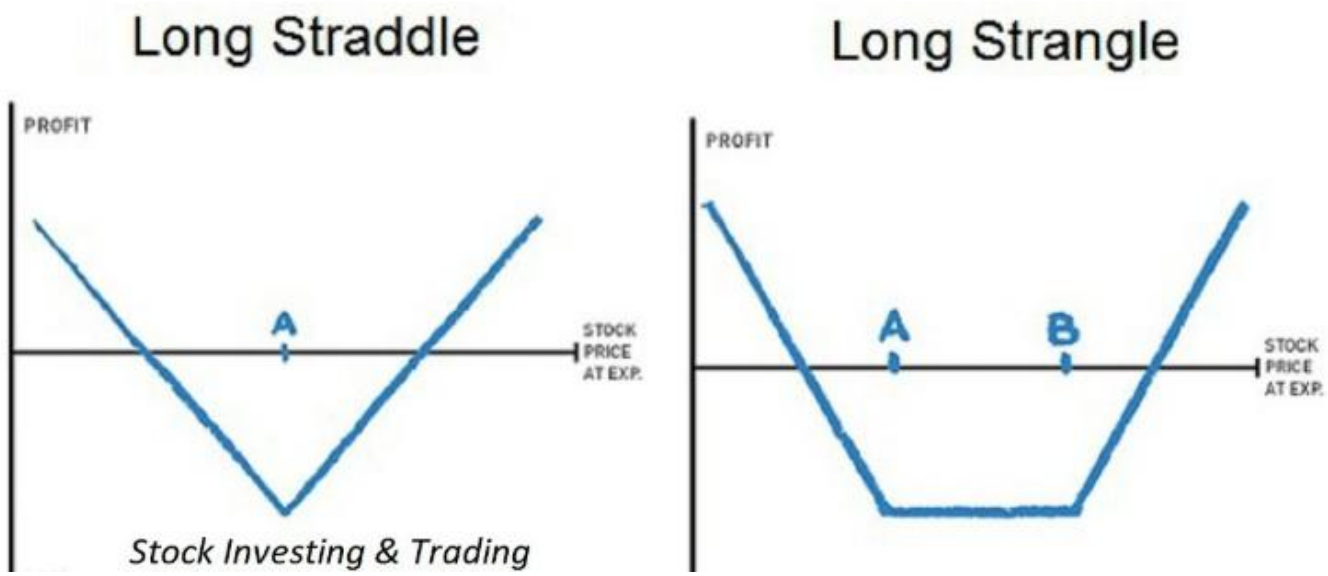
The evaluation remains subject to snapshot expiration prices, simplified execution and the exclusion of early exercise, assignment and mark-to-market losses. The unresolved position is excluded from reported profit, and the cumulative profit chart represents realised trade outcomes rather than account equity. The system should therefore be understood as a decision-support tool that assists strategy analysis rather than guarantees trading success. Further evaluation on new dates is required to assess whether the observed behaviour persists beyond the research period.

9. Future Improvements

Future development could improve the system's coverage, recommendation quality and reliability. The following enhancements focus on supporting a wider range of market expectations, refining the reasoning process and presenting more informative outcome estimates.

9.1 Expand Market Data and Strategy Coverage

The system currently supports six directional strategies and accepts either a bullish or bearish outlook. Future versions could include neutral and volatility-based strategies to address a wider range of expectations. For example, a range-bound outlook could support an Iron Condor, while an expectation that the price will finish near a target could support a Butterfly Spread. A large movement expected in either direction could support a Long Straddle or Long Strangle, provided the movement is sufficient to overcome the premiums paid.



Additional market information, such as earnings dates, dividend announcements and changes in implied volatility, could provide context beyond the existing indicators. The interface could also collect the user's expected movement size, target range and investment horizon. Supporting these additions would require updates to the knowledge graph, reasoning rules, payoff engine and genetic algorithm. Each new strategy should be evaluated using consistent cost and risk assumptions, with clear explanations of its suitability and potential losses.

9.2 Strengthen the Recommendation Rules

The current system uses predefined scores to rank strategies according to market conditions. These rules could be evaluated across more stocks and market periods to determine whether they produce consistent and suitable recommendations.

Additional rules could consider expected price movement, upcoming events and user risk preferences. The knowledge graph could also link each recommendation directly to the observations, thresholds and score adjustments responsible for its ranking. This would improve consistency between the graph's conceptual relationships and the implemented reasoning process.

Minimum suitability requirements could be introduced before the final ranking. These could assess cost-adjusted expected profit, maximum loss, liquidity and data quality. If no strategy satisfies the requirements, the system could return "No suitable strategy under the current conditions" and explain the main reasons. This would address the limitation that the highest-ranked strategy may still have an unfavourable expected outcome. New rules and thresholds should be validated on separate data to avoid tailoring them too closely to the development dataset.

9.3 Improve Outcome Estimates and Uncertainty Reporting

The current Monte Carlo model uses simplified assumptions about future prices and volatility. Future development could assess alternative models and examine how recommendations change under different assumptions about price movement, volatility and transaction costs.

Probability of profit should be recalculated after explicit transaction costs so that it is consistent with the adjusted expected profit and payoff curve. The system could also display the average gain in profitable scenarios and the average loss in unprofitable scenarios. These measures would help explain why a high probability of profit does not necessarily imply positive expected profit. Recommendations could be evaluated across several plausible scenarios to show how sensitive their rankings and expected outcomes are to modelling assumptions. The interface could identify results that remain relatively stable and those that change substantially.

Finally, estimated probabilities could be compared with subsequently observed outcomes across a larger set of recommendations. This would help identify systematic optimism or pessimism and improve the reliability of the information presented to users.

10. Conclusion

This project developed an Options Strategy Recommendation and Optimisation System that integrates market-data preparation, feature engineering, knowledge representation, rule-based reasoning and genetic optimisation. The system accepts a stock ticker and a bullish or bearish outlook, then evaluates supported options strategies using observed market conditions and defined decision objectives.

The knowledge graph represents relationships between directional outlook, market characteristics and strategy families. Explicit reasoning rules rank strategies according to volatility conditions, trend alignment, liquidity and data availability. The payoff engine estimates strategy outcomes, while the genetic algorithm balances cost-adjusted expectancy, probability of profit, downside protection, capital efficiency, liquidity and reasoning alignment. A Streamlit application presents the recommendation, explanations, payoff profile and evaluation evidence.

Historical evaluation using SPY options data from 2020 to 2022 combined the recommendation engine with a chronological trading policy. The saved results recorded 47 completed trades, an 80.85% win rate and a net profit of $1,342.08, with one additional selected position remaining unresolved. Performance varied across periods, and the 2022 retrospective test period produced a loss despite the positive aggregate result.

These findings illustrate that a strategy recommendation is only part of a trading decision. Entry timing determines whether market conditions support acting on the recommendation, while risk management controls the exposure accepted. The reported outcomes reflect these components operating together and do not establish the independent contribution of each component.

The project also provided practical experience in combining intelligent reasoning with quantitative evaluation. It demonstrates how explicit domain knowledge and optimisation can support a transparent, repeatable decision process while keeping market uncertainty and risk central to the interpretation of its recommendations.

11. References

1. Aroussi, R. (n.d.) *yfinance documentation*. Available at: https://ranaroussi.github.io/yfinance/

2. Graupe, K. (n.d.) *$SPY option chains – Q1 2020 – Q4 2022* [Dataset]. Kaggle. Available at: https://www.kaggle.com/datasets/kylegraupe/spy-daily-eod-options-quotes-2020-2022

3. Murphy, J.J. (1999) *Technical analysis of the financial markets: A comprehensive guide to

trading methods and applications*. New York, NY: New York Institute of Finance. Available at: https://www.penguin.com.au/books/technical-analysis-of-the-financial-markets-9781101659199

4. The Options Playbook (n.d.) *Long call*. Available at: https://www.optionsplaybook.com/option-strategies/long-call

5. The Options Playbook (n.d.) *Long call spread*. Available at: https://www.optionsplaybook.com/option-strategies/long-call-spread

6. The Options Playbook (n.d.) *Short put spread*. Available at: https://www.optionsplaybook.com/option-strategies/short-put-spread

7. The Options Playbook (n.d.) *Long put*. Available at: https://www.optionsplaybook.com/option-strategies/long-put

8. The Options Playbook (n.d.) *Long put spread*. Available at: https://www.optionsplaybook.com/option-strategies/long-put-spread

9. The Options Playbook (n.d.) *Short call spread*. Available at: https://www.optionsplaybook.com/option-strategies/short-call-spread

10. Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S. and Zimmermann, A. (2021) 'Knowledge graphs', *ACM Computing Surveys*, 54(4), Article 71, pp. 1–37. Available at: https://doi.org/10.1145/3447772

11. Glasserman, P. (2003) *Monte Carlo methods in financial engineering*. New York, NY: Springer. Available at: https://doi.org/10.1007/978-0-387-21617-1

12. Marler, R.T. and Arora, J.S. (2010) 'The weighted sum method for multi-objective optimization: New insights', *Structural and Multidisciplinary Optimization*, 41, pp. 853–862. Available at: https://doi.org/10.1007/s00158-009-0460-7

13. Mitchell, M. (1996) *An introduction to genetic algorithms*. Cambridge, MA: MIT Press. Available at: https://doi.org/10.7551/mitpress/3927.001.0001

14. Streamlit (n.d.) *Streamlit documentation*. Available at: https://docs.streamlit.io/

15. OpenAI. (2026). ChatGPT (August 14 version) [Large language model]. https://chat.openai.com